

Homework for class 2: OOPS, Async & Pydantic,

Python OOP — Homework

1. Variadic Arguments: `args` and `*kwargs`

Homework: Flexible AI Model Configuration

Create a function:

```
def configure_model(*args, **kwargs):  
    ...
```

The function should:

- Accept any number of model features using `args`
- Accept model configuration using `*kwargs`
- Print all features and configuration values in a readable format

Example:

```
configure_model(  
    "RAG",  
    "Tool Calling",  
    "Streaming",  
    model="gpt-5",  
    temperature=0.2,  
    max_tokens=1000  
)
```

Expected output should clearly separate the positional features from the keyword-based configuration.

2. Classes & Object-Oriented Programming

Homework: AI Model Class

Create an `AIModel` class with:

- `name`
- `provider`
- `model_type`

Add a method:

```
describe()
```

that prints information about the model.

Create at least **three different** `AIModel` **objects** and call `describe()` on each.

3. Dynamic Instance Attributes & Lifecycle

Homework: AI Agent Session

Create an `AgentSession` class with a constructor that accepts:

```
user_id
```

Initially, the object should only contain `user_id`.

Then dynamically add these attributes after creating the object:

```
model  
message_count  
is_active
```

Add methods:

```
start()
```

```
end()
```

`start()` should activate the session and `end()` should deactivate it.

Also investigate and demonstrate what happens when an object is deleted using:

```
del session
```

Use `__del__()` to observe when the object's lifecycle ends.

4. Class Methods

Homework: AI Model Factory

Create an `AIModel` class with:

```
name
provider
```

Add a class method:

```
from_string()
```

that can create an `AIModel` from a string like:

```
"GPT-5:OpenAI"
```

Example:

```
model = AIModel.from_string("GPT-5:OpenAI")

print(model.name)
print(model.provider)
```

Expected:

GPT-5
OpenAI

The method should use `@classmethod` and `cls` rather than directly referencing `AIModel`.

5. Inheritance & `super()`

Homework: AI Engineer Hierarchy

Create a base class:

```
class SoftwareEngineer:  
    ...
```

with:

- `name`
- `experience`

and a method:

```
describe()
```

Create a child class:

```
class AIEngineer(SoftwareEngineer):  
    ...
```

Add:

- `specialization`

Override `describe()` in `AIEngineer`, but use:

```
super().describe()
```

to reuse the parent implementation.

Create an `AIEngineer` object and demonstrate how the child class extends the behavior of the parent class.

Async Await Homework

Homework 1: Async Functions and `await`

Create an async function called `process_task()` that simulates a task taking 2 seconds to complete.

Inside the function:

1. Print `"Task started"`
2. Wait asynchronously for 2 seconds using `asyncio.sleep()`
3. Print `"Task completed"`
4. Return `"Success"`

Create an `async main()` function that calls `process_task()` and prints the returned result.

Expected Output

```
Task started
Task completed
Success
```

Goal

Understand:

- `async def`

- `await`
 - `asyncio.sleep()`
 - How an async function is executed using `asyncio.run()`
-

Homework 2: Concurrency with `asyncio.gather()`

You have three independent tasks:

- Downloading a document → 3 seconds
- Generating embeddings → 2 seconds
- Fetching metadata → 1 second

Create three async functions that simulate these operations using `asyncio.sleep()`.

Then use `asyncio.gather()` to execute all three tasks concurrently.

Measure the total execution time using `time.perf_counter()`.

Requirements

Print the result of each task and the total execution time.

Expected Observation

If the tasks were executed sequentially, the total time would be approximately:

```
3 + 2 + 1 = 6 seconds
```

With `asyncio.gather()`, the total time should be approximately:

```
3 seconds
```

Goal

Understand how asynchronous concurrency can reduce the total waiting time for independent I/O-bound operations.

Homework 3: Concurrent API Requests

Use `httpx.AsyncClient` and the JSONPlaceholder API.

Fetch the following three resources concurrently:

```
<https://jsonplaceholder.typicode.com/users/1>  
<https://jsonplaceholder.typicode.com/posts/1>  
<https://jsonplaceholder.typicode.com/todos/1>
```

Requirements

1. Create an async function called `fetch_data(url)`.
2. Use `httpx.AsyncClient` to make the HTTP request.
3. Return the JSON response.
4. Use `asyncio.gather()` to call all three URLs concurrently.
5. Print the important information from each response.

For example:

```
User: Leanne Graham  
Post ID: 1  
Todo: delectus aut autem
```

Goal

Combine the concepts you have learned:

- `async def`
- `await`
- `httpx.AsyncClient`
- `asyncio.gather()`
- Concurrent I/O

Homework 4: Build an Async API Client

Create a small `PostService` class that communicates with the JSONPlaceholder API. The class should contain an async method:

```
async def get_post(self, post_id):  
    ...
```

The method should:

1. Accept a `post_id`.
2. Make an asynchronous HTTP request using `httpx.AsyncClient`.
3. Fetch the corresponding post from:

```
<https://jsonplaceholder.typicode.com/posts/{post_id}>
```

1. Return the JSON response.

Then fetch posts with IDs `1`, `2`, and `3` concurrently using `asyncio.gather()`.

Expected Output

Print something similar to:

```
Post 1: ...  
Post 2: ...  
Post 3: ...
```

Bonus

Add basic error handling:

- If the API returns a non-success status code, raise an exception.
- Handle the exception in `main()` and print a meaningful error message.

Goal

Practice structuring asynchronous HTTP code into a reusable service, similar to how you will later interact with LLM APIs.

Homework 5: Build a Concurrent LLM-Style Data Fetcher

Imagine that your AI application needs information from three independent sources before calling an LLM:

- User profile
- User's posts
- User's todos

Use the JSONPlaceholder API to simulate these three data sources.

Create three async functions:

```
async def get_user():
    ...

async def get_posts():
    ...

async def get_todos():
    ...
```

Each function should make an HTTP request using `httpx.AsyncClient`.

Then create an `async main()` function that:

1. Starts all three requests concurrently using `asyncio.gather()`.
2. Waits for all three responses.
3. Combines the results into a single Python dictionary.

The final structure should look similar to:

```
{
    "user": {...},
    "posts": [...],
```

```
"todos": [...]  
}
```

Print the final dictionary.

Bonus

Measure the total execution time and compare it with a sequential implementation.

Goal

Think about how async concurrency is used in real AI systems, where an application may need to fetch multiple pieces of information before sending a request to an LLM.

Pydantic Homework

Homework 1: User Profile Model

Create a Pydantic `User` model with:

- `name`
- `age`
- `email`
- Optional `phone`
- `password` using `SecretStr`

Create an instance and practice **direct attribute access**.

Also serialize the model and deserialize it back into a Pydantic object.

Homework 2: Nested Order Model

Create Pydantic models for:

- `Customer`
- `Address`
- `Product`
- `Order`

An `Order` should contain a nested `Customer`, an `Address`, and a collection of `Product` objects.

Validate the input and access nested fields using normal attribute access.

Homework 3: Validated API Request

Create a `UserRequest` model with:

- Required and optional fields
- An aliased field using `Field`
- Constraints such as minimum/maximum length and numeric limits
- `Annotated` for at least one field

Add a custom validator for one business rule.

Homework 4: Application Configuration

Create a Pydantic model representing application configuration.

Include:

- API key using `SecretStr`
- Environment/name fields
- A dictionary of model configuration
- Appropriate field constraints
- At least one optional field

Serialize the configuration and verify that sensitive values are handled appropriately.

Homework 5: LLM Response Pipeline

Create a Pydantic model representing a structured LLM response:

- `answer`
- `confidence`
- `sources`
- `metadata`

Add appropriate validation constraints and one custom validation rule.

Simulate the complete pipeline:

```
JSON Response
↓
Pydantic Validation
↓
Python Object
↓
Application Logic
↓
Serialization
↓
JSON
```